



Machine Learning-Based Network Intrusion Detection System

Summer Bachelor Semester Project
Student: João Bernardo Sousa Faria
Supervisor: Salima Lamsiyah

Objective

Problem:

- Cyberattacks are frequent
- Network traffic is large
- Manual detection is difficult

Goal: Detect malicious traffic and classify attack types using machine learning (ML).

- False negatives miss attacks
- False positives create alert fatigue (analysts may become overwhelmed by too many incorrect warnings)



Dataset

CICIDS2017 network-flow dataset

- Loaded from 8 CICIDS2017 CSV files
- Contains labelled network traffic flows (each row is one flow, each column describes a characteristic of the same flow)
- 300,000 sampled flows from approximately 2.8 million available flows
- 69 numeric features
- 15 labels

Dataset Imbalance

- 254,282 benign flows vs 45,718 attack flows (imbalance ratio = 5.56:1)



Destination	Flow Durat	Total Fwd F	Total Back	Flow Bytes	Flow Packe	SYN Flag	C ACK Flag	C Label	
16813	68	1	1	176470.6	29411.8	0	0	BENIGN	
53	184	2	2	1152174	21739.1	0	0	BENIGN	
443	10268877	7	5	184.6	1.17	0	1	BENIGN	
80	1293792	3	7	4302.5	7.73	1	0	DDoS	
444	109	1	1	110091.7	18348.6	1	0	PortScan	
21	534205	8	6	2890.4	26.2	0	1	FTP-Patator	



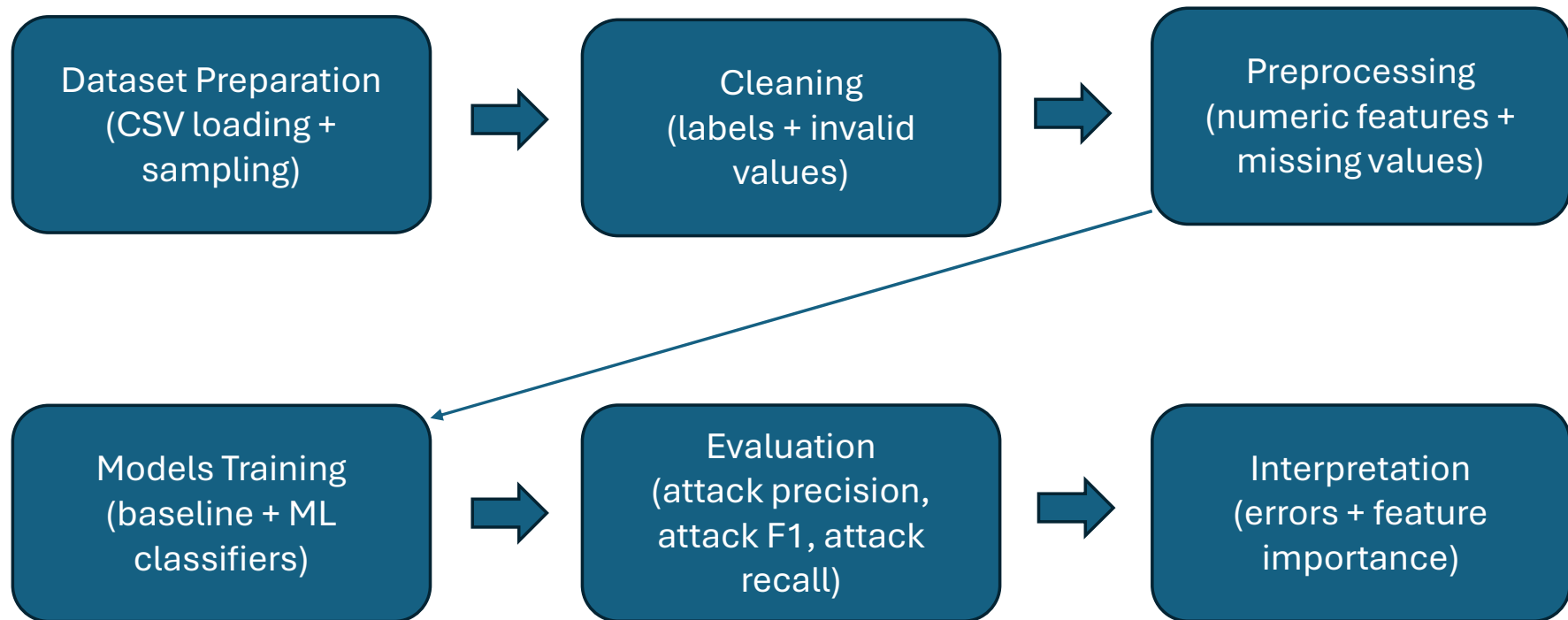
Attack Types

Through Binary intrusion Detection (BENIGN or ATTACK) and Multiclass attack classification

Frequent attacks:

- DDoS (distributed traffic flood)
- PortScan (open-port discovery)
- DoS variants (service disruption attacks)
- Bot (automated infected-host activity)
- Web Attacks (application-layer attacks)
- Infiltration (unauthorized internal access)

System Pipeline





Models Comparison

- HistGradientBoosting (boosted tree model optimized for structured data)
- Tuned Random Forest (Random Forest with optimized parameters)
- Random Forest (ensemble of decision trees)
- Logistic Regression (linear classification model)
- Rule baseline (simple manually defined detection rules)
- Dummy baseline (naive model that predicts the majority class)

Main Binary Results

model	accuracy	precision_attack	recall_attack	f1_attack
HistGradientBoosting	0.998507	0.994754	0.995451	0.995102
Tuned Random Forest	0.998240	0.994485	0.993963	0.994224
Random Forest	0.998173	0.994050	0.993963	0.994007
Logistic Regression	0.930907	0.700745	0.954068	0.808017
Rule-Based Baseline	0.859493	0.567576	0.327734	0.415530
Dummy Majority	0.847600	0.000000	0.000000	0.000000

Why Accuracy Is Not Enough

model	accuracy	precision_attack	recall_attack	f1_attack
Dummy Majority	0.847600	0.000000	0.000000	0.000000

- Dummy Majority always predicts the most common class (in our case, it is BENIGN)
- Most flows are benign, therefore the accuracy is high.
- Therefore, the model never predicts ATTACK, which explains the neutral values for attack evaluation metrics.
- **Takeaway:** A model can have high accuracy due to the dataset imbalance but not be linked to the intrusion detection task.

Cross-Validation Results

model	mean_test_f1_attack	std_test_f1_attack
HistGradientBoosting	0.992935	0.001304
Random Forest	0.988472	0.000766
Logistic Regression	0.815847	0.002462

Goal: Check whether each model performs consistently across different data splits

- 3-fold cross-validation: each model was trained/tested 3 times
- Mean F1 shows average attack detection performance
- Low standard deviation shows stable performance across folds

Source-File Holdout

Why Source-File Holdout?

- More realistic test using CSV files unseen during training
- Checks whether the model generalizes beyond the original random split

model	accuracy	precision_attack	recall_attack	f1_attack
HistGradientBoosting - Source File Holdout	0.895450	0.996375	0.607911	0.755112
Random Forest - Source File Holdout	0.895170	0.994736	0.607861	0.754602
Logistic Regression - Source File Holdout	0.865931	0.837775	0.613088	0.708033

Key Finding

- HistGradientBoosting performed best on unseen source files
- Recall dropped compared to the random split, showing this test is harder
- High precision means most predicted attacks were truly attacks

False Positives + False Negatives

False Positive

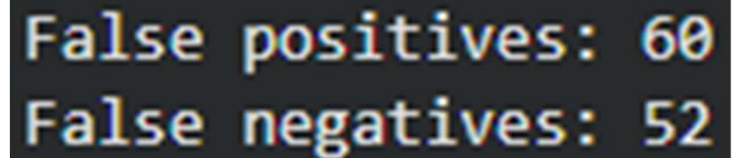
- Benign traffic: predicted as ATTACK
- Count: 60
- Impact: unnecessary alerts

False Negative

- Attack traffic: predicted as BENIGN
- Count: 52
- Impact: missed attacks

Takeaway

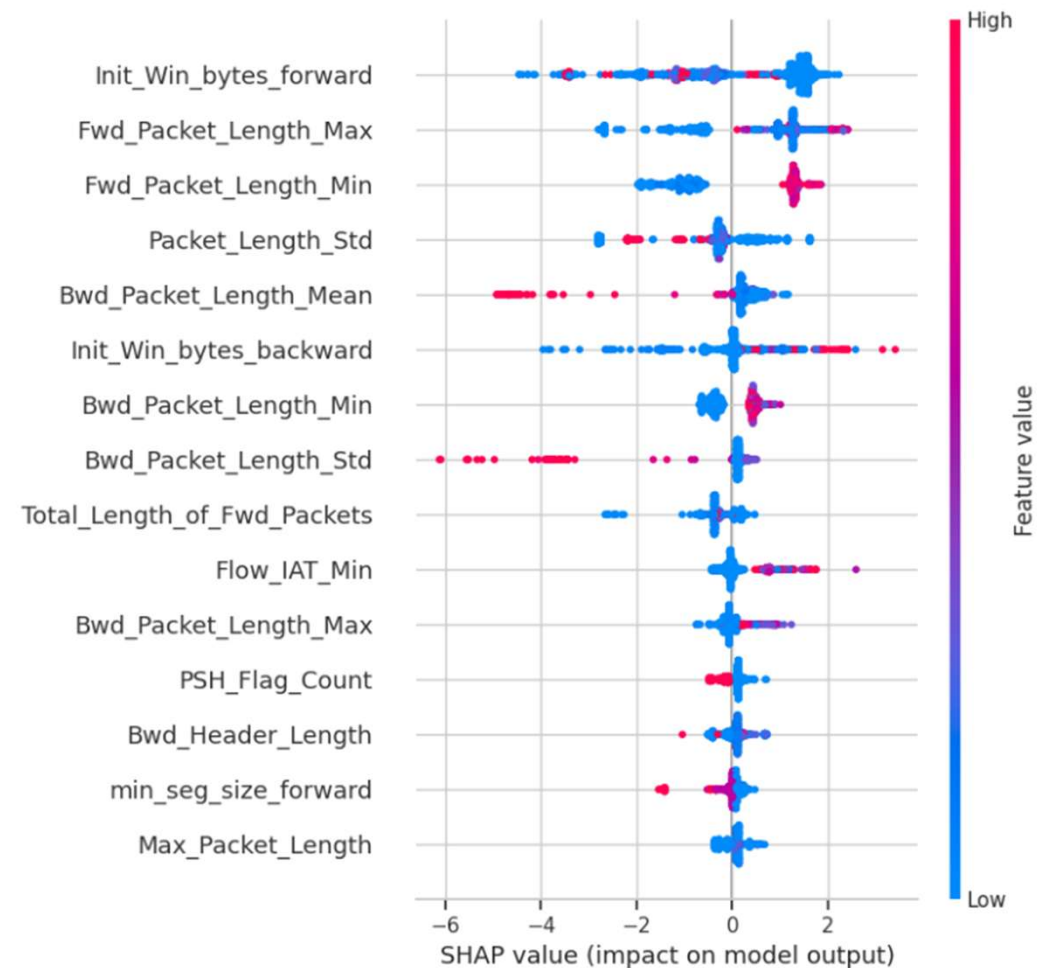
Errors are low on the random test split, but false negatives are the most critical IDS risk.



False positives: 60
False negatives: 52

SHAP Explainability

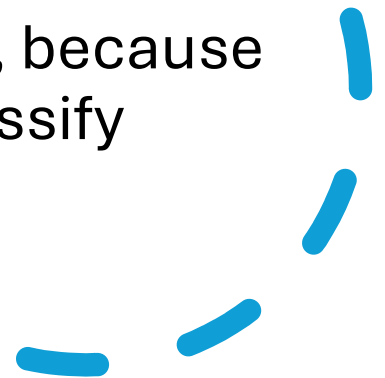
- **Goal:** understand how the best model makes its predictions
- Applied to HistGradientBoosting
- Identifies the most important network-flow features
- Adds transparency to the IDS decision-making process



task	objective	best_model	main_metric	score
Binary detection	BENIGN vs ATTACK	HistGradientBoosting	attack F1	0.995253
Multiclass classification	Specific attack category	HistGradientBoosting	macro F1	0.147532
Multiclass classification	Specific attack category	HistGradientBoosting	weighted F1	0.845060

Multiclass Classification

- Binary detection is strong, but multiclass macro F1 is lower, because rare attacks are harder to classify (class imbalance)



Conclusion

- Developed an ML-based IDS prototype using CICIDS2017
- Best binary model: HistGradientBoosting
- Attack F1-score: 0.995
- Source-file holdout showed weaker generalization
- Attack F1-score dropped to 0.759
- Class imbalance and rare attacks affected evaluation
- ML is promising, but realistic testing is essential

Limitations + Future Work

Limitations

- Sampled dataset
- Offline prototype
- Class imbalance

Future Work

- Full dataset run
- Stronger temporal/file split
- Better handling of class imbalance
- Live traffic prototype





Thank you